Lập trình mạng · Chương 4

Bài toán ứng dụng: xử lý đơn hàng qua message broker

TS. Đặng Ngọc Hùng

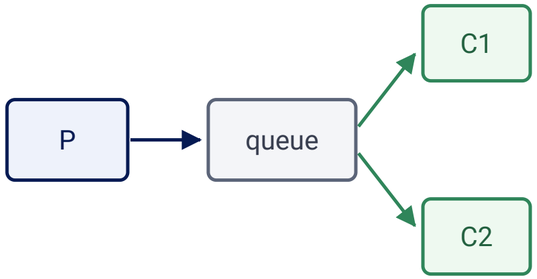
Một sự kiện đặt hàng, ba nhu cầu khác nhau

1. Thống kê doanh số: **mọi** đơn hàng, một bên nhận
2. Gửi email xác nhận: mọi đơn hàng, việc chậm, **chia** cho nhiều worker
3. Kho Hà Nội: **chỉ** đơn hàng giao ở Hà Nội

> **Bài toán:** một producer, ba nhóm consumer; chọn mẫu giao tiếp và dựng exchange, queue, binding cho từng nhu cầu.

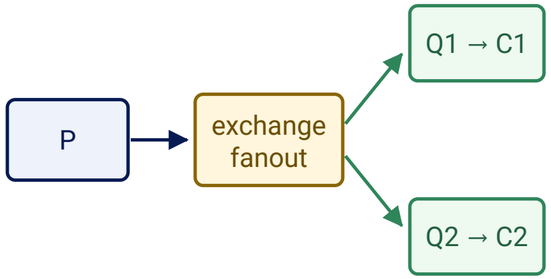
Bốn mẫu giao tiếp qua broker

work queue



mỗi message một consumer:
chia việc

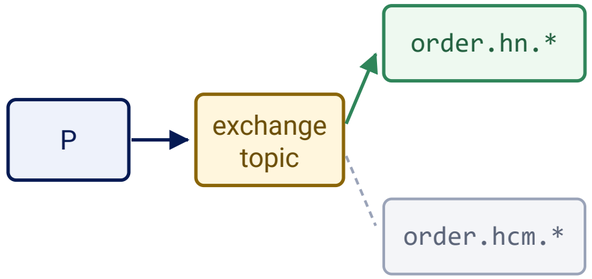
publish/subscribe



mỗi bên đăng ký một queue:
ai cũng nhận một bản

routing theo topic

`order.hn.1001`



chọn queue theo mẫu
của routing key

RPC qua AMQP

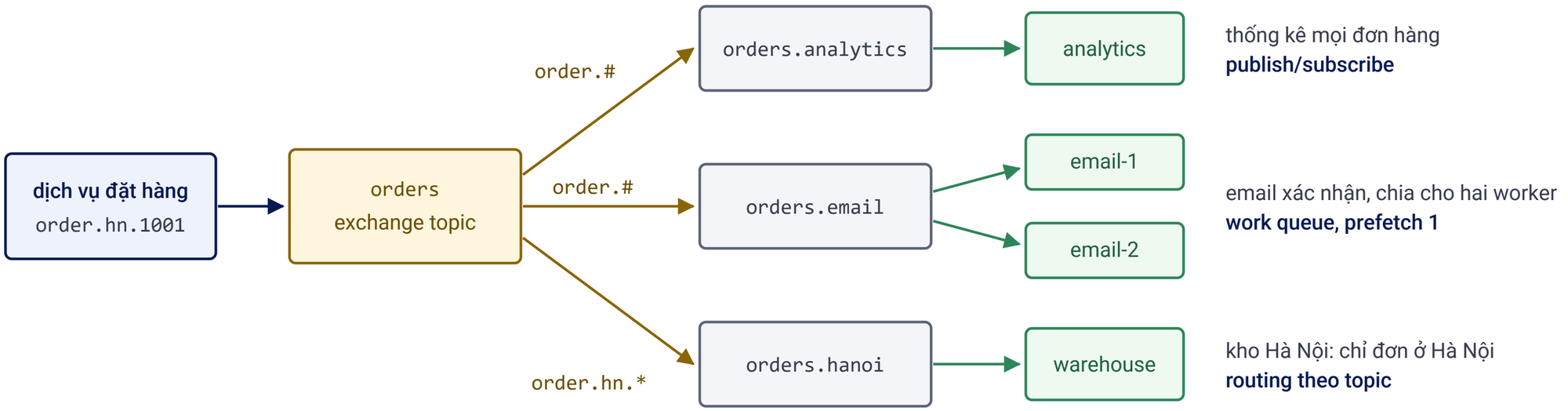


yêu cầu kèm reply-to
và correlation ID;
kết quả về queue reply-to,
ghép bằng correlation ID

Mở rộng

> Các mẫu dựng từ exchange, queue, binding: **work queue** chia việc, **publish/subscribe** phát cho mọi bên, **routing theo topic** chọn theo mẫu của **routing key**.

Một exchange topic, ba queue cho ba nhu cầu



> Producer gửi mỗi sự kiện **một lần**; binding và số consumer trên mỗi queue quyết định mẫu giao tiếp.

Ba mẫu chỉ khác nhau ở binding

```java
channel.exchangeDeclare(EXCHANGE, BuiltinExchangeType.TOPIC);
channel.queueBind("orders.analytics", EXCHANGE, "order.#");  // mọi đơn hàng: publish/subscribe
channel.queueBind("orders.email", EXCHANGE, "order.#");      // mọi đơn hàng, chia cho các worker: work queue
channel.queueBind("orders.hanoi", EXCHANGE, "order.hn.*");   // chỉ đơn ở Hà Nội: routing theo topic
```

- Ba mẫu khác nhau ở **binding** và số consumer trên mỗi queue
- Thêm một bộ phận nhận: thêm queue và binding, **không sửa producer**

Một sự kiện tới ba nhóm consumer; email chia cho hai worker

Java

```
consume(connection, "email-1", "orders.email", prefetch, 600, emailDone);  // worker chậm
consume(connection, "email-2", "orders.email", prefetch, 100, emailDone);  // worker nhanh
channel.basicQos(prefetch);  // tối đa prefetch message chưa ack giao cho consumer này
```

Kết quả chạy

```
$ OrderDemo --prefetch 1
published 6 orders, email prefetch 1
  analytics 6: 1001 1002 1003 1004 1005 1006
  warehouse 4: 1001 1002 1004 1006
  email-1   1: 1001
  email-2   5: 1002 1003 1004 1005 1006
  all email sent after 625 ms
```

Không giới hạn prefetch: worker chậm giữ việc trong khi worker nhanh rảnh

Java

```
channel.basicQos(prefetch);  // tối đa prefetch message chưa ack giao cho consumer này
```

Kết quả chạy

```
$ OrderDemo
published 6 orders, email prefetch unlimited
  analytics 6: 1001 1002 1003 1004 1005 1006
  warehouse 4: 1001 1002 1004 1006
  email-1   3: 1001 1003 1005
  email-2   3: 1002 1004 1006
  all email sent after 1905 ms
```

- Không giới hạn: broker chia đều 3 và 3 ngay khi giao; worker chậm giữ 3 email, xong sau **1,9 giây**
- **Prefetch** bằng 1: message chưa giao còn trong queue, worker rảnh nhận trước

Câu hỏi 1

Ba hệ thống, mỗi hệ thống cần nhận một bản của mọi sự kiện đặt hàng. Nên dựng exchange và queue thế nào?

A. Một queue chung, ba hệ thống cùng consume từ queue đó.

B. Exchange direct với routing key bằng tên hệ thống.

C. Exchange fanout, mỗi hệ thống một queue riêng bind vào exchange.

D. Ba producer, mỗi producer gửi cho một hệ thống.

Câu hỏi 1

Ba hệ thống, mỗi hệ thống cần nhận một bản của mọi sự kiện đặt hàng. Nên dựng exchange và queue thế nào?

A. Một queue chung, ba hệ thống cùng consume từ queue đó.

B. Exchange direct với routing key bằng tên hệ thống.

C. Exchange fanout, mỗi hệ thống một queue riêng bind vào exchange.

D. Ba producer, mỗi producer gửi cho một hệ thống.

Publish/subscribe: mỗi bên đăng ký một queue riêng; exchange fanout gửi một bản tới mọi queue đã bind. Nhiều consumer trên một queue là work queue, mỗi message chỉ một consumer nhận.

Câu hỏi 2

Trong RPC qua AMQP, client gửi nhiều yêu cầu rồi nhận kết quả ở cùng một queue reply-to. Correlation ID dùng để làm gì?

A. Ghép mỗi kết quả với đúng yêu cầu đã gửi, vì kết quả có thể về không theo thứ tự.

B. Mã hóa message để broker không đọc được.

C. Chọn exchange cho yêu cầu.

D. Báo broker xóa message sau khi xử lý.

Câu hỏi 2

Trong RPC qua AMQP, client gửi nhiều yêu cầu rồi nhận kết quả ở cùng một queue reply-to. Correlation ID dùng để làm gì?

A. Ghép mỗi kết quả với đúng yêu cầu đã gửi, vì kết quả có thể về không theo thứ tự.

B. Mã hóa message để broker không đọc được.

C. Chọn exchange cho yêu cầu.

D. Báo broker xóa message sau khi xử lý.

Các kết quả cùng về queue reply-to và có thể về không theo thứ tự gửi; correlation ID gắn ở cả yêu cầu và kết quả để client ghép đúng cặp.

Tổng kết

1. **Work queue**: một queue, nhiều worker chia việc; đặt **prefetch** nhỏ để worker rảnh nhận trước

2. **Publish/subscribe**: mỗi bên một queue, mỗi queue một bản của mọi message

3. **Routing theo topic**: chọn queue theo mẫu của **routing key** (* một từ, # không hoặc nhiều từ)

Từ khóa cần nhớ work queue · publish/subscribe · routing key · routing theo topic · prefetch

Mở rộng

Chủ đề nâng cao

- RPC qua AMQP: queue reply-to và correlation ID
- So sánh RabbitMQ với Kafka

Bài tập mở rộng

Dựng RPC qua AMQP theo RabbitMQ Tutorial 6: server tính số Fibonacci, client nhận kết quả ở queue reply-to, ghép bằng correlation ID.

Tổng kết chương 4

1. **RPC**: gọi phương thức từ xa như cục bộ với Java RMI; truyền tham số, idempotency key, truy cập đồng thời
2. **Web service**: SOAP với WSDL và XML; REST với tài nguyên, phương thức HTTP và JSON
3. **gRPC**: hợp đồng .proto, Protocol Buffers trên HTTP/2; streaming, deadline, mã trạng thái
4. **Message broker**: giao tiếp bất đồng bộ qua exchange, queue, binding; ack; các mẫu giao tiếp

Tiếp theo

Bảo mật: TLS và secure socket trong Java